

1. Rappel du sujet

Application mobile (Expo / React Native) pour gérer un stock d'aliments, leurs lots et dates de péremption. L'objectif est de suivre les produits, détecter les expirations imminentes, et conserver un historique des lots consommés ou jetés.

2. Specifications des choix

Tableau de bord avec acces rapide aux sections principales. Gestion des aliments: creation, consultation, modification, suppression. Gestion des lots: ajout d'un lot (date, quantite), marquage consomme ou jete, suppression d'un lot. Alertes de peremption: seuils globaux (J-x) activables/desactivables. Alertes personnalisées par aliment: seuils spécifique a un aliment qui remplacent les globaux. Liste des alertes avec tri par urgence et details par alerte. Historique des lots archives (consommés/jetés) avec detail. Recherche d'aliments par nom. Scanner de code-barres pour retrouver un aliment existant ou en creer un. Persistence locale via SQLite, migrations et schema versionne.

3. Justification technique

- Expo: dev rapide, beaucoup de librairies, recommande par la team React Native (framework avec routing file-based, libs universelles, plugins natifs).
- Expo Router: navigation file-based simple a maintenir, ecrans decoupees par fonctionnalite.
- Drizzle ORM + Expo SQLite: ORM type-safe et dev rapide, base locale sans backend, migrations claires.
- NativeWind (Tailwind): styles declaratifs, rapidite d'iteration, coherence visuelle.
- Expo Camera + DateTimePicker: moins de lignes de code, scan codes-barres et selection de dates sans tout faire soi-meme.

4. Architecture

- app/: routes Expo Router (tabs, ecrans details, settings, alerts, history).
- features/: logique par domaine (scanner, food-new, food-id, alerts).
- components/: UI reutilisable (cards, buttons, inputs, layout, empty state).
- hooks/: acces donnees via Drizzle (CRUD aliments, lots, alertes, recettes).
- db/: schema, provider SQLite, migrations Drizzle.
- constants/: constantes UI (couleurs).

Flux de donnees typique: Screen -> Feature Provider (ex: FoodContext) -> hooks/useDatabase -> Drizzle -> SQLite -> mise a jour UI.

5. Auto-evaluation

| Fonctionnalite annoncee | Realisee | Commentaire || — | — | — || Gérer les aliments (ajout, modification, suppression, consultation) | Oui | CRUD complet. || Sélectionner des aliments en scannant le code-barres | Oui | Redirection vers fiche ou creation. || Gérer des recettes | Non | Ecrans presents, pas de logique. || Enregistrer date de peremption, quantite, etc | Oui | Lots avec date + quantite. || Alertes generales pour tous les aliments (J-7, J-3, J-1) | Oui | Seuils globaux configurables. || Alertes personnalisées par aliment (prioritaires sur globales) | Oui | Seuils custom actifs. || Historique des produits consommés | Oui | Statut consomme + vue historique. || Historique des produits jetés | Oui | Statut jete + vue historique. || Suggestions de recettes selon aliments proches de peremption | Non | Non implemente. || Recherche et filtres | Oui | Recherche par nom + filtres historique. || Tableau de bord | Oui | Acces aux sections. |